

Courier Tracking — Tasarım ve Üretim Hazırlık Analizi

Bartuğ Sevindik · Haziran 2026

Bu notta, çözümü neden bu şekilde tasarladığımı ve gerçek bir üretim ortamına nasıl taşıyacağımı anlatıyorum. Vaka metninde "**streaming geolocations**" ve "**scalable web application**" ifadeleri geçtiği için, problemi tek seferlik bir ödev gibi değil; sade ama büyümeye hazır bir iskelet gibi ele aldım.

1. Bugünkü çözüm (teslim ettiğim sürüm)

- **Mesafe:** Haversine formülü, bir `DistanceCalculator` arayüzünün (Strategy) arkasında.
- **Olay dağıtımı:** Observer — `CourierLocationPublisher` tek konum olayını `StoreEntranceListener` ve `TravelDistanceListener` 'a dağıtıyor.
- **1 dakika kuralı:** Olayın *zaman damgasına* göre, `ConcurrentHashMap.compute(...)` ile atomik.
- **Mağazalar:** `stores.json` açılışta bir kez okunuyor (`@PostConstruct`), bellekte salt-okunur.
- **Durum:** Bellek içi, eşzamanlı erişime uygun yapılarla.

5 mağaza için bu $O(N)$ yaklaşımı tamamen yeterli ve harici bağımlılık gerektirmeden çalışıyor.

2. Ölçeklenince: coğrafi indeksleme (Redis GEO)

"Ya 5 değil de 50.000 mağaza olsaydı?" sorusunun cevabı bende hazır. O ölçekte her kurye hareketinde tüm mağazaları taramak ($O(N)$) sistemi yorar. Bu durumda mağazaları açılışta Redis GEO'ya yükler (`opsForGeo().add`), yakınlık sorgusunu `GEOSEARCH / GEORADIUS` ile yapardım — yaklaşık $O(\log N)$, 100 metrelik yarıçapı milisaniyeler içinde tarar. Ayrıca Haversine'i Java'da elle hesaplamak yerine işi Redis'in kendi motoruna bırakmış olurdum; bu daha performanslı.

Bu deseni daha önce taksi durağı / şube yakınlık sistemlerinde ve POS ısı haritası projesinde kullandım; benim için tanıdık bir yapı. Önemli nokta şu: mesafe hesabını Strategy arkasına aldığım için, bu geçiş iş mantığının geri kalanını değiştirmeden yapılabilir — sadece `DistanceCalculator` 'ın yeni bir gerçekleştirimi.

3. 1 dakika kuralını dağıtık ortamda nasıl çözerim?

Tek instance'ta olay-zamanı + `ConcurrentHashMap` yeterli ve deterministik. Ama uygulama yatayda ölçeklenip birden çok kopya çalışırsa, bellek içi map kopyalar arasında paylaşılmaz. O zaman kuralı Redis'e taşırdım — `SET NX + TTL` ile:

```
String logKey = "courier-entrance:" + courierId + ":" + storeId;
Boolean ilkGiris = redisTemplate.opsForValue()
    .setIfAbsent(logKey, "ENTERED", Duration.ofMinutes(1));

if (Boolean.TRUE.equals(ilkGiris)) {
    // ilk giriş: logla ve kalıcı kaydı at
} else {
    // 1 dakika içinde tekrar geldi: atla
}
```

Redis'in `setIfAbsent` (NX) + TTL ikilisi, dağıtık sistemlerde debouncing / rate-limiting için atomik ve doğru bir yöntemdir; tüm kopyalar aynı anahtarı paylaşır.

4. Toplam mesafeyi dağıtık ortamda

Bu projede `CourierState` 'i bellekte `compute` ile atomik güncelliyorum. Ölçekte ise kuryenin son konumunu (`last-loc:courierId`) ve toplam mesafesini (`total-dist:courierId`) Redis'te tutar; her yeni konumda bir önceki konumla arasını `GEODIST` ile hesaplar ve toplama atomik olarak (`INCRBYFLOAT`) eklerdim.

5. Tasarım desenleri

- **Strategy** — `DistanceCalculator` : bugün Haversine, yarın Redis GEO veya bir harita servisi. Kullanan kod sabit kalır (Open/Closed, Dependency Inversion).
- **Observer** — tek bir konum olayından iki bağımsız iş (giriş tespiti, mesafe birikimi) tetikleniyor; her dinleyici tek sorumluluk taşıyor, yeni bir tepki eklemek için yeni bir dinleyici yeter.

6. Özet

Teslim ettiğim sürüm, vakanın istediği her şeyi karşılıyor ve kendi başına çalışıyor. Mesafe hesabını ve olay akışını soyutladığım için; ölçeklenme, dağıtık debouncing ve dağıtık mesafe takibi gibi senaryolara geçiş gerektiğinde iş mantığının geri kalanını yeniden yazmam gerekmez.